README

HelixCode - Distributed AI Development Platform

HelixCode

A distributed, AI-powered software development platform with multi-platform support.

Features

- **Multi-Platform Support**: Desktop, mobile, terminal, and specialized OS clients
- **Distributed Computing**: Worker nodes for parallel task execution
- **AI Integration**: LLM-powered code generation and reasoning with multiple free providers
- **Free AI Models**: Access to XAI (Grok), OpenRouter, GitHub Copilot, and Qwen without API keys
- **Cognee.ai Memory Integration**: Advanced memory management with knowledge graphs, semantic search, and real-time processing
- **Real-time Collaboration**: MCP protocol for tool execution
- **Authentication & Security**: JWT-based auth with session management
- **Task Management**: Checkpoint-based work preservation
- **Notification System**: Multi-channel notifications (Slack, Email, Discord)

Quick Start

Development

```
# Clone the repository
git clone https://github.com/HelixDevelopment/Helix-CLI.git
cd helixcode
```

```
# Install dependencies
go mod download

# Generate assets
make logo-assets

# Build the server
make build

# Run tests
make test

# Start development server
make dev
```

Production Deployment

1. **Clone and setup:**

   ```
   git clone https://github.com/HelixDevelopment/Helix-CLI.git
   cd helixcode
   cp .env.example .env
   ```

2. **Configure environment variables:** Edit `.env` file with your production values:

   ```
   HELIX_AUTH_JWT_SECRET=your-super-secure-jwt-secret
   HELIX_DATABASE_PASSWORD=your-secure-database-password
   HELIX_REDIS_PASSWORD=your-secure-redis-password
   ```

3. **Deploy with Docker Compose:**

   ```
   docker-compose up -d
   ```

4. **Check deployment:**

   ```
   docker-compose ps
   curl http://localhost/health
   ```

Architecture

Core Components

- **Server**: Main API server with REST and WebSocket endpoints
- **Database**: PostgreSQL for persistent data storage

- **Cache**: Redis for session and task state management
- **Workers**: Distributed worker nodes for task execution
- **MCP Server**: Model Context Protocol for AI tool integration

AI Providers

HelixCode supports multiple AI providers with a focus on free and accessible models, plus enterprise-grade premium providers with advanced features:

Free Providers (No API Key Required)

- **XAI (Grok)**: `grok-3-fast-beta`, `grok-3-mini-fast-beta`, `grok-3-beta` - Fast and capable models
- **OpenRouter**: `openai/gpt-oss-20b:free`, `meta-llama/llama-3.2-3b-instruct:free` - Free models from various providers
- **GitHub Copilot**: `gpt-4o`, `claude-3.5-sonnet`, `claude-3.7-sonnet`, `o1`, `gemini-2.0-flash` - Free with GitHub subscription
- **Qwen**: OAuth2 authentication available, 2,000 requests/day free tier

Premium Providers (Advanced Features)

Anthropic Claude NEW

The most powerful coding assistant with industry-leading reasoning capabilities: - **Models**: Claude 4 Sonnet/Opus, Claude 3.7 Sonnet, Claude 3.5 Sonnet/Haiku, Claude 3 Opus/Sonnet/Haiku - **Context Windows**: 200K tokens (all models) - **Max Output**: Up to 50K tokens (Claude 4/3.7) - **Advanced Features**: - **Extended Thinking**: Automatic reasoning mode for complex problems - **Prompt Caching**: Up to 90% cost reduction on repeated contexts - **Tool Caching**: Cache tool definitions for multi-turn conversations - **Vision Support**: Analyze images and diagrams - **Streaming**: Real-time token-by-token responses

Google Gemini NEW

Google's most capable AI models with massive context windows: - **Models**: Gemini 2.5 Pro/Flash, Gemini 2.0 Flash, Gemini 1.5 Pro/Flash - **Context Windows**: Up to 2M tokens (Gemini 2.5 Pro, 1.5 Pro) - **Max Output**: 8K tokens - **Advanced Features**: - **Massive Context**: Handle entire codebases (2M tokens = ~1.5M words) - **Multimodal**: Text, images, and code understanding - **Flash Models**: Ultra-fast responses with 1M token context - **Function Calling**: Native tool integration - **Safety Controls**: Configurable content filtering

OpenAI

Industry-standard models with broad ecosystem support: - **Models**: GPT-4.1, GPT-4.5 Preview, GPT-4o, O1/O3 (reasoning), O4 Mini - **Context Windows**: Up to 1M+ tokens (GPT-4.1) - **Max Output**: Variable by model - **Features**: Function calling, vision support, reasoning models

Local Models

- **Ollama**: Run any GGUF model locally
- **Llama.cpp**: Direct llama.cpp integration
- **Privacy**: 100% offline, no data leaves your machine

Applications

- **Desktop**: Full-featured desktop application (Fyne)
- **Terminal UI**: Terminal-based interface (tview)
- **Aurora OS**: Specialized Aurora OS client
- **Harmony OS**: Specialized Harmony OS client
- **Mobile**: Cross-platform mobile applications

Configuration

Configuration is managed through YAML files and environment variables. See `config/config.yaml` for default settings.

Key configuration areas: - Server settings (ports, timeouts) - Database connection - Redis configuration - Authentication settings - Worker management - LLM provider settings (XAI, OpenRouter, Copilot, Qwen OAuth2)

Getting Started with Free AI

HelixCode comes with multiple free AI providers pre-configured:

Quick AI Setup

```
# Use XAI (Grok) - no setup required
helixcode llm provider set xai

# Use OpenRouter free models
helixcode llm provider set openrouter

# Use GitHub Copilot (requires GitHub token)
export GITHUB_TOKEN="your_github_token"
helixcode llm provider set copilot
```

```
# Use Qwen with OAuth2 (interactive setup)
helixcode llm auth qwen
```

Environment Variables for All Providers

Free Providers:

```
# GitHub Copilot
export GITHUB_TOKEN="ghp_your_github_token"

# OpenRouter (optional, for higher rate limits)
export OPENROUTER_API_KEY="sk-or-your-key"

# XAI (optional, for higher rate limits)
export XAI_API_KEY="xai-your-key"
```

Premium Providers:

```
# Anthropic Claude
export ANTHROPIC_API_KEY="sk-ant-your-key"

# Google Gemini
export GEMINI_API_KEY="your-gemini-key"
# or
export GOOGLE_API_KEY="your-google-key"

# OpenAI
export OPENAI_API_KEY="sk-your-openai-key"
```

Quick Setup for New Providers

Anthropic Claude (with Extended Thinking & Prompt Caching):

```
# Set API key
export ANTHROPIC_API_KEY="sk-ant-your-key-here"

# Use Claude 4 Sonnet (most powerful)
helixcode llm provider set anthropic --model claude-4-sonnet

# Or use Claude 3.5 Sonnet (best for coding)
helixcode llm provider set anthropic --model claude-3-5-sonnet-
        latest
```

```
# Generate code with extended thinking
helixcode generate "Think carefully: design a distributed cache
        system"
```

Google Gemini (with 2M token context):

```
# Set API key
export GEMINI_API_KEY="your-gemini-key"

# Use Gemini 2.5 Pro (2M context)
helixcode llm provider set gemini --model gemini-2.5-pro

# Or use Gemini 2.5 Flash (fast with 1M context)
helixcode llm provider set gemini --model gemini-2.5-flash

# Process entire codebase
helixcode analyze --full-context --model gemini-2.5-pro
```

API Documentation

Authentication Endpoints

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/refresh` - Token refresh
- `GET /api/auth/me` - Current user info

Task Management

- `GET /api/tasks` - List tasks
- `POST /api/tasks` - Create task
- `GET /api/tasks/{id}` - Get task details
- `PUT /api/tasks/{id}` - Update task
- `DELETE /api/tasks/{id}` - Delete task

Worker Management

- `GET /api/workers` - List workers
- `POST /api/workers` - Register worker
- `GET /api/workers/{id}` - Get worker details
- `DELETE /api/workers/{id}` - Remove worker

Development

Building Applications

```
# Build all applications
make prod

# Build specific applications
make aurora-os
make harmony-os

# Build mobile bindings
make mobile-ios
make mobile-android
```

Testing

Test Infrastructure (MANDATORY)

Before running integration tests, you **MUST** start the test infrastructure containers. These provide PostgreSQL, Redis, ChromaDB, Qdrant, Cognee, and Ollama services needed for full test coverage.

Start Test Infrastructure:

```
# Using Docker Compose
docker compose -f docker-compose.test.yml up -d

# OR using Podman
podman-compose -f docker-compose.test.yml up -d

# Wait for all services to be healthy (check status)
docker compose -f docker-compose.test.yml ps
# OR
podman-compose -f docker-compose.test.yml ps
```

Test Infrastructure Services: | Service | Port | Purpose | |———|——|———| | postgres-test | 5433 | PostgreSQL database for persistence tests | | redis-test | 6380 | Redis for caching and session tests | | chromadb-test | 8002 | ChromaDB vector storage tests | | qdrant-test | 6333/6334 | Qdrant vector database tests | | cognee-test | 8001 | Cognee.ai memory integration tests | | ollama-test | 11434 | Local LLM integration tests |

Stop Test Infrastructure:

```
docker compose -f docker-compose.test.yml down -v
# OR
podman-compose -f docker-compose.test.yml down -v
```

Running Tests

```
# Run all tests (requires test infrastructure)
make test

# Run unit tests only (no infrastructure needed)
./run_tests.sh

# Run all tests including integration and e2e
./run_all_tests.sh

# Run specific test suites
go test ./internal/auth/...
go test ./internal/worker/...

# Run with coverage
go test -cover ./...

# Run with coverage report
make test-coverage
```

Code Quality

```
# Format code
make fmt

# Lint code
make lint
```

Deployment Options

Docker Compose (Recommended)

The production `docker-compose.yml` includes: - HelixCode server - PostgreSQL database - Redis cache - Nginx reverse proxy - Prometheus monitoring - Grafana dashboards

Manual Deployment

1. Build the binary: `make prod`

2. Setup PostgreSQL and Redis
3. Configure environment variables
4. Run the server: `./bin/helixcode-server`

Kubernetes

For large-scale deployments, use the provided Kubernetes manifests in the `k8s/` directory.

Monitoring

The deployment includes Prometheus and Grafana for monitoring: - Application metrics - Database performance - Worker health - Task execution stats

Access Grafana at `http://localhost:3000` (default credentials: admin/admin)

Security

- JWT-based authentication
- Password hashing with bcrypt
- SSH key-based worker authentication
- Environment variable configuration
- No secrets in code or config files

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests
5. Submit a pull request

License

This project is licensed under the terms specified in the LICENSE file.

Documentation

- TUI Capabilities Guide — streaming, MCP tools, LSP diagnostics, skills, plugins, the agentic read-only tool loop, the Helix Agent ensemble (visible members + per-member model via LLMsVerifier), environment providers, and the HelixAgent full-capacity provider, each with a real trigger and example prompt.

Support

For support and questions: - GitHub Issues: https://github.com/HelixDevelopment/Helix-CLI/issues - Documentation: https://docs.helixcode.dev - Community: https://community.helixcode.dev